

Кратко в предыдущих сериях

Советуем перед семинаром освежить в памяти из курса статистики:

- Описательные статистики: среднее, частота абсолютная и относительная, выборочные медиана и квантили, их поиск с помощью пакета `pymru`
- ЦПТ
- Точечные и интервальные оценки
- Оценки параметров среднего и пропорции
- Визуализации с помощью `matplotlib` и `seaborn`
- Работа с датасетами в `pandas`

Тема 2. Как правильно доносить информацию.

0. Цель и задачи

Цель — имея готовые инсайты из исследования, научиться визуализировать и презентовать их так, чтобы в продукт были внесены изменения.

Задачи:

- потренироваться с выбором визуализации
- улучшить визуализацию
- ознакомиться с интерактивными визуализациями с помощью `plotly` и `dash`
- изучить метод донесения информации с помощью фреймворка STAR, опробовать его на практике

1. Вспомним, что было на лекции про визуализации

Вопрос в зал: какие основные принципы хорошей визуализации?

Студенты перечисляют список с лекции

- **Графическая целостность и честность визуализации:** масштабы правильные, оси верны, подписи понятны, не было "подгона" данных
- **Соотношение данных и чернил:** визуализация не перегружена, понятна, из нее легко сделать выводы
- **Многофункциональность графических элементов:** график читается без пояснений, тренды и закономерности легко видны

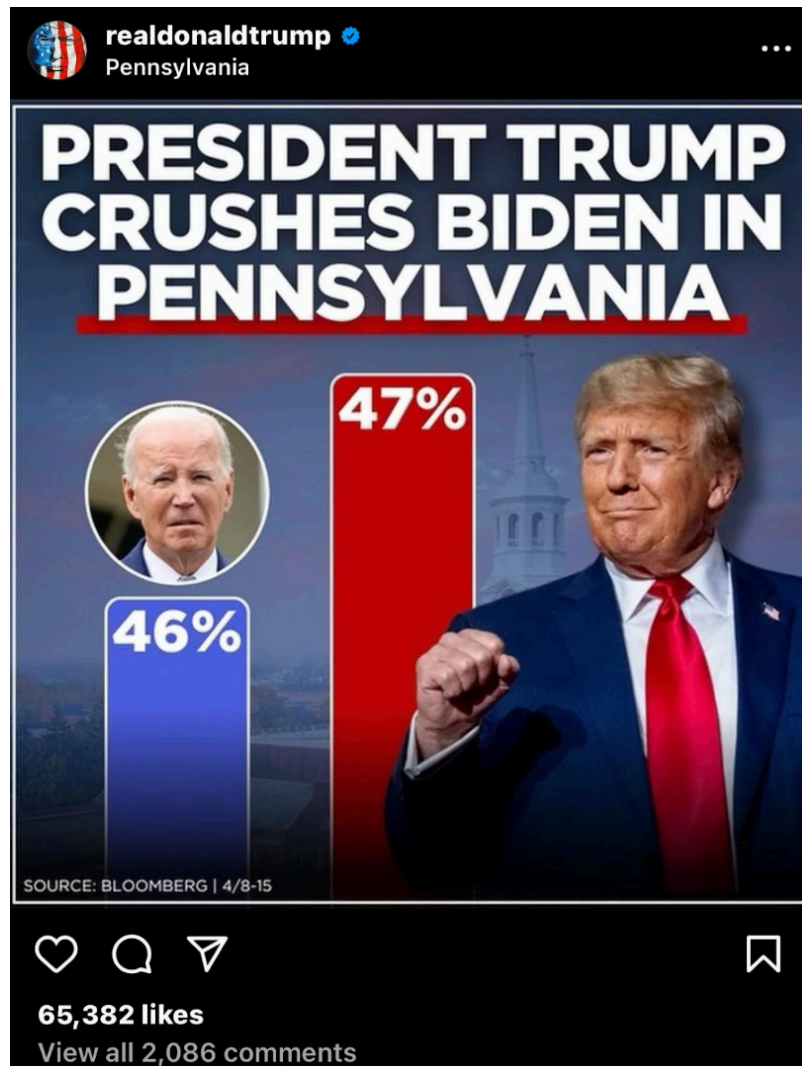
2. Улучшаем визуализации

2.1 Какая визуализация лучше

На восприятие работы влияет качество визуализации результатов. Плохая структура или неподходящий формат убьют все попытки в качественную аналитику.

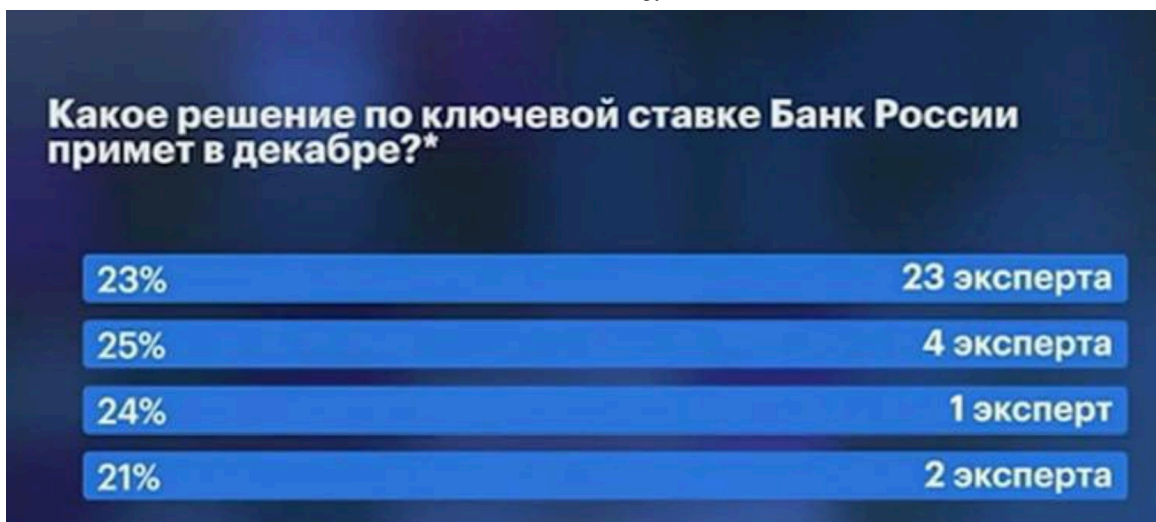
Примеры:

0.1 Без комментариев



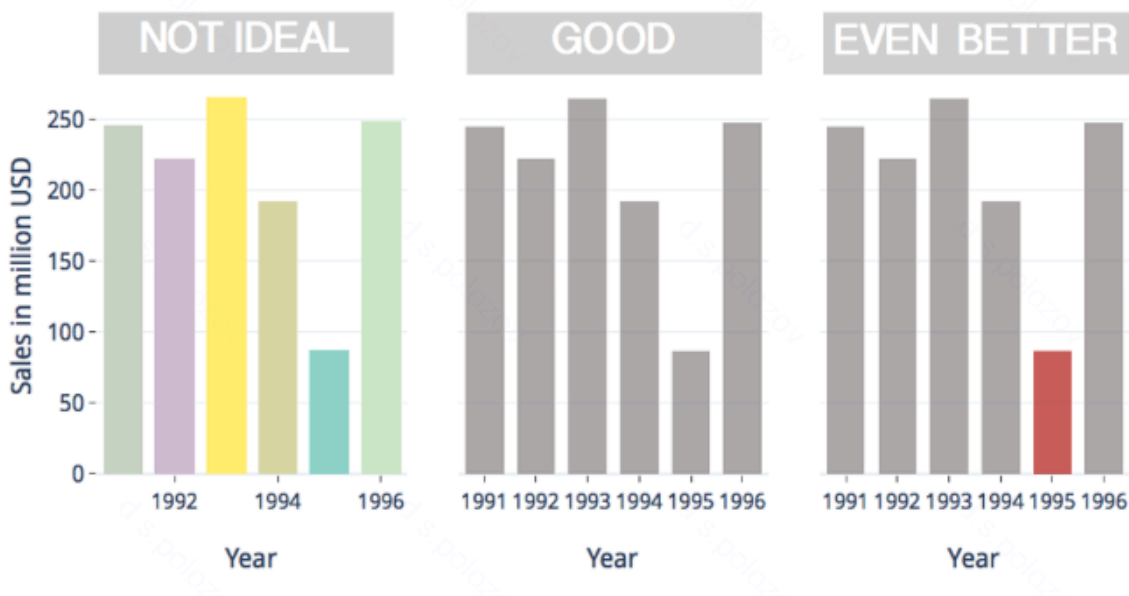
- ❌ *Плохо:* явная "нечестность" визуализации

0.2 Угадайте, по какой логике отсортированы столбцы



1. Дискотека цветов

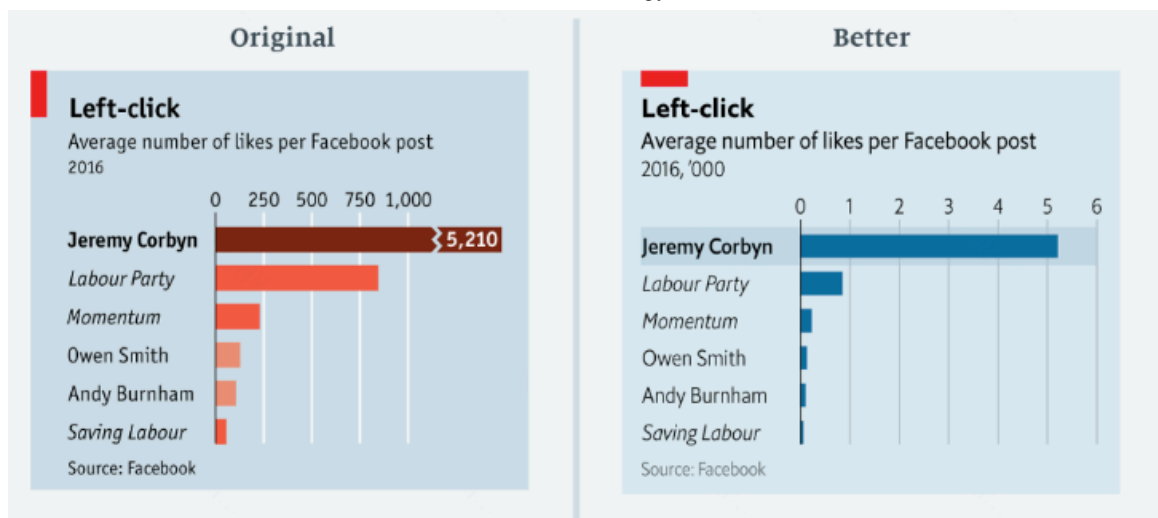
Гистограммы продаж



- ✗ *Плохая*: визуализация перегружена: хаотичные цвета, неполные подписи, сложность сравнения
- ✓ *Хорошая*: единая цветовая гамма, подписи на все года, акцент на важные данные

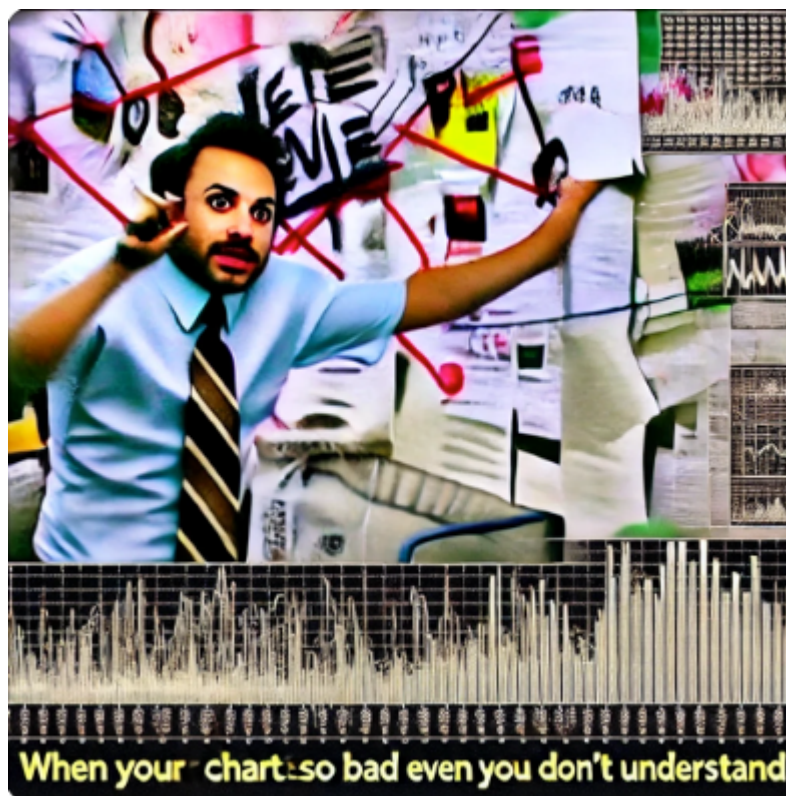
2. Что с масштабом 🤔

Среднее число лайков

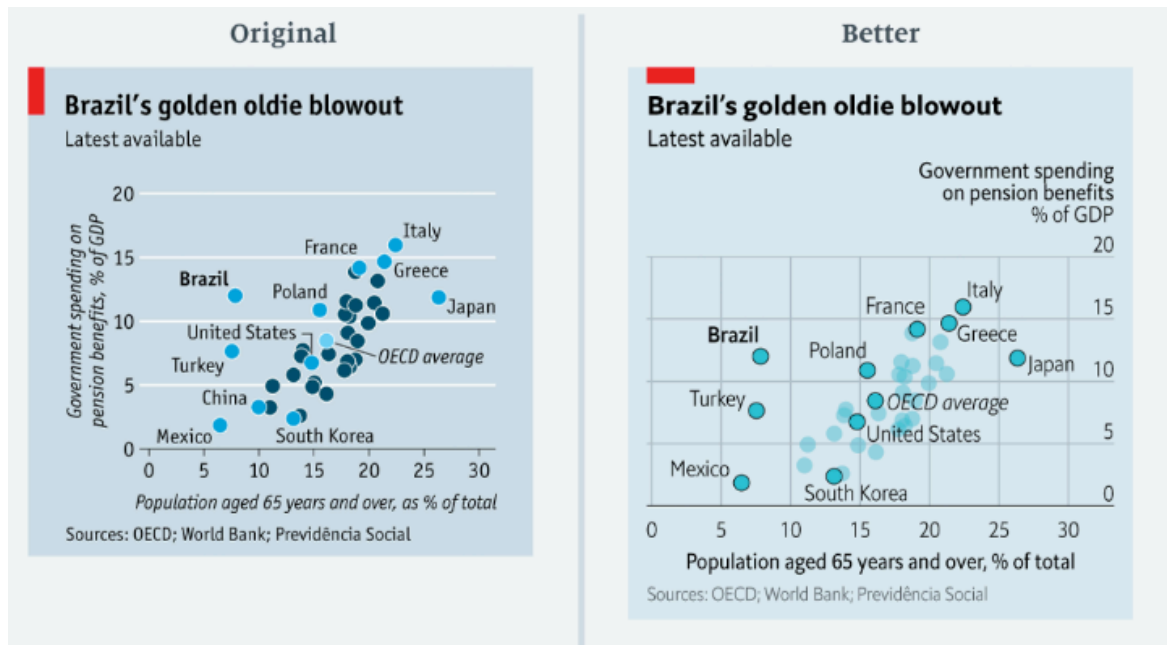


- ❌ *Плохая*: масштаб и абсолютные значения искажают восприятие, непонятно, что означают цвета
- ✅ *Хорошая*: пропорциональный масштаб, удобные числовые обозначения, четкий акцент

3. Зачем столько информации?



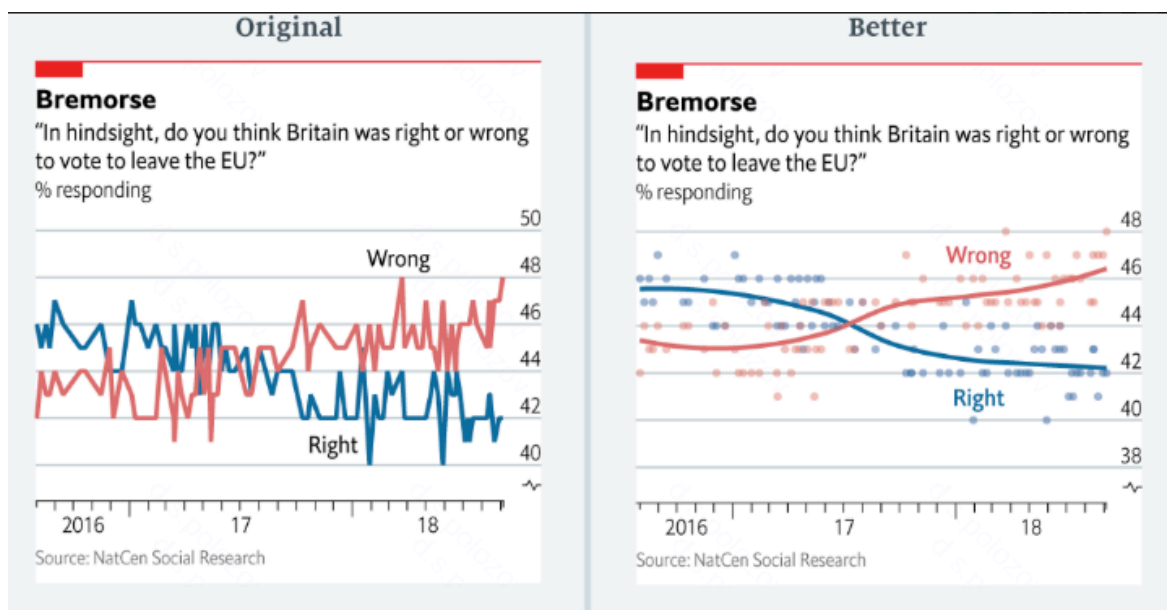
Госрасходы на пенсии



- ✗ *Плохая:* перегруженность, плохо читаемые точки
- ✓ *Хорошая:* оптимизированный размер и цвет точек, логичный масштаб

4. Качаемся на качелях

Опрос по Brexit

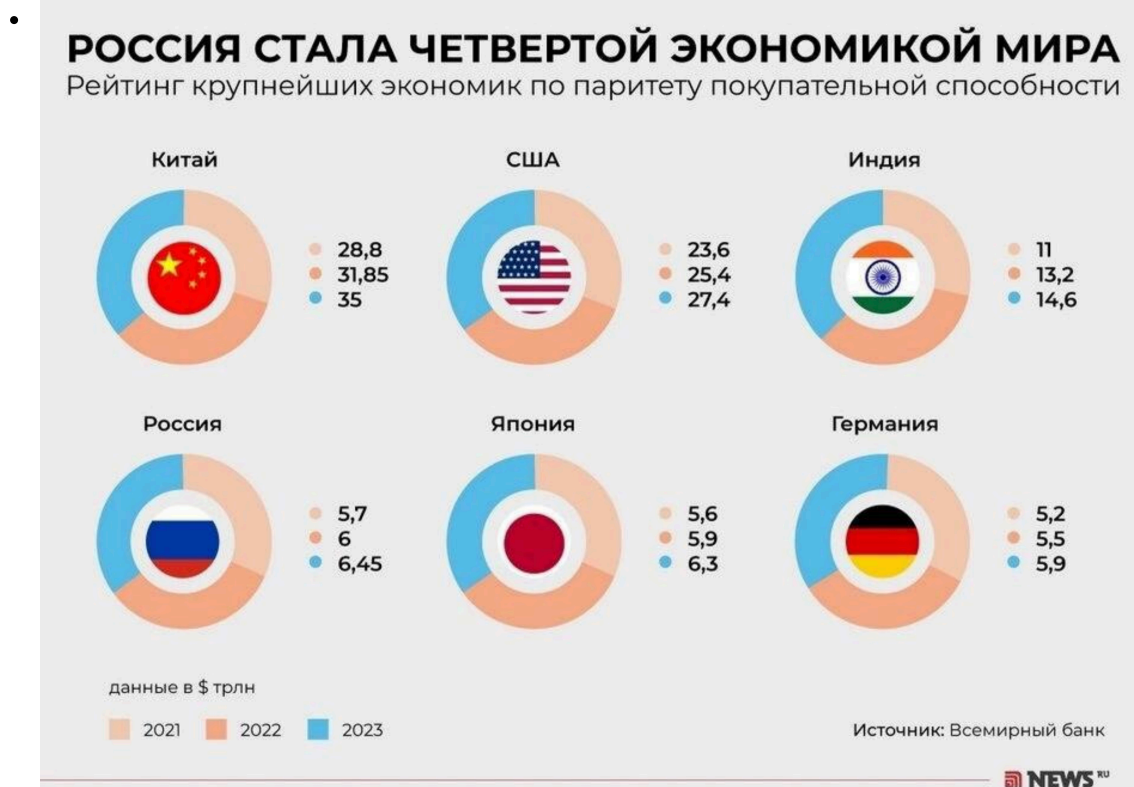


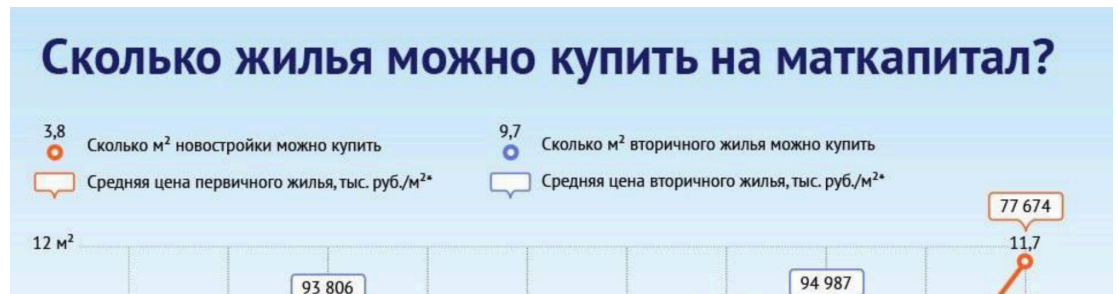
- ✗ *Плохая:* резкие линии, мешающие увидеть тренд
- ✓ *Хорошая:* сглаженные тренды, точки данных, четкая цветовая кодировка

Итого:

Хорошие визуализации делают данные понятными, убирают лишний шум и

2.2 Что не так? Как лучше?





2.2. Кейс на улучшение визуализации

Ввод [21]:

```
1 import matplotlib.pyplot as plt
2 import scipy.stats as sps
3 import numpy as np
4 import seaborn as sns
5 import pandas as pd
6 sns.set_theme(style='whitegrid')
```

Загрузка данных

Используем датасет **Titanic**, встроенный в Seaborn.

Ввод [2]:

```
1 # Загружаем данные
2 df = sns.load_dataset('titanic')
3 df.head()
```

Out [2]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True

Описание датасета Titanic

Этот датасет содержит информацию о пассажирах печально известного лайнера *Титаник*, который затонул в 1912 году.

Данные включают характеристики пассажиров, такие как их возраст, пол, класс каюты, а также информацию о выживании.

Описание столбцов:

Колонка	Тип данных	Описание
survived	int (0/1)	Выжил ли пассажир (0 = нет, 1 = да)
pclass	int	Класс каюты пассажира (1 = первый, 2 = второй, 3 = третий)
sex	object	Пол пассажира (male или female)
age	float	Возраст пассажира в годах
sibsp	int	Количество братьев/сестер или супругов на борту

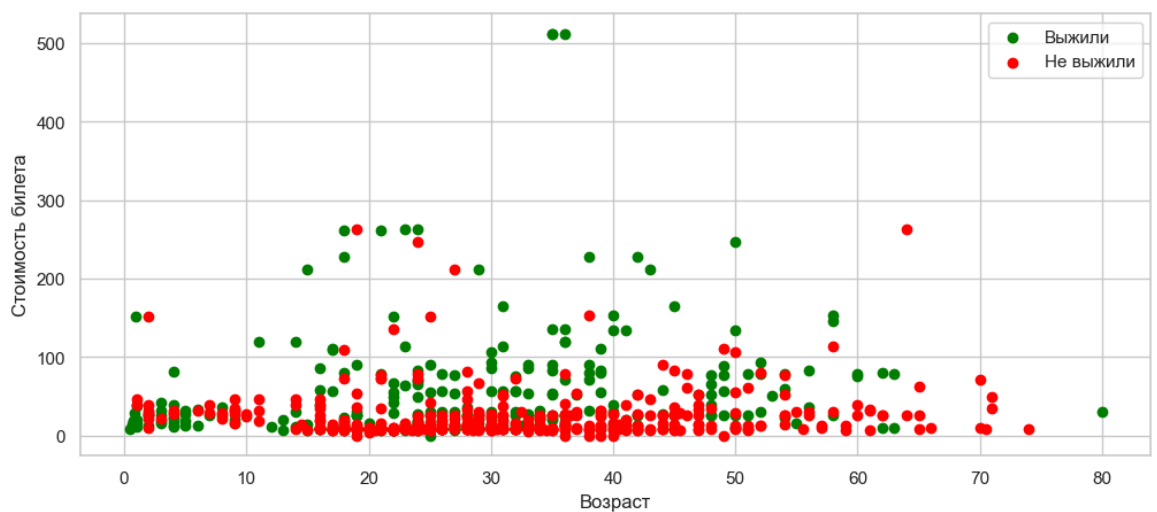
Колонка	Тип данных	Описание
parch	int	Количество родителей/детей на борту
fare	float	Стоимость билета
embarked	object	Порт посадки (C = Cherbourg, Q = Queenstown, S = Southampton)
class	object	Альтернативное обозначение класса каюты (First , Second , Third)
who	object	Категория пассажира (man , woman , child)
adult_male	bool	Является ли пассажир взрослым мужчиной (True/False)
deck	object	Буква палубы каюты (A , B , C и т. д.)
embark_town	object	Название города, где пассажир сел на борт
alive	object	Выжил ли пассажир (yes / no)

Ввод [3]:

```

1 df_0 = df[df['survived'] == 0]
2 df_1 = df[df['survived'] == 1]
3
4 plt.figure(figsize=(12, 5))
5 plt.scatter(x=df_1['age'], y=df_1['fare'], c='green', label='Выжили')
6 plt.scatter(x=df_0['age'], y=df_0['fare'], c='red', label='Не выжили')
7 plt.xlabel('Возраст')
8 plt.ylabel('Стоимость билета')
9 plt.legend()
10 plt.show()

```



Дорабатываем график

- Что не так с графиком?
- Доработайте график
- Сделайте вывод о зависимости возраста и стоимости билета

Что не так:

- Слишком много больших и жирных точек, которые накладываются друг на друга
- Непонятна зависимость: кажется, что точки хаотичны
- Можно увеличить масштаб

```

Ввод [4]: 1 df['group_age'] = df['age'] // 5
          2
          3 df['group_age'] = df['group_age'].apply(lambda group: 14. if group
          4 df['group_age'] *= 5
          5 df['group_age'] += 2.5
          6 df_groupby_age = df.groupby(['group_age', 'survived'])['fare'].mea
          7 df_groupby_age.head(5)

```

Out [4]:

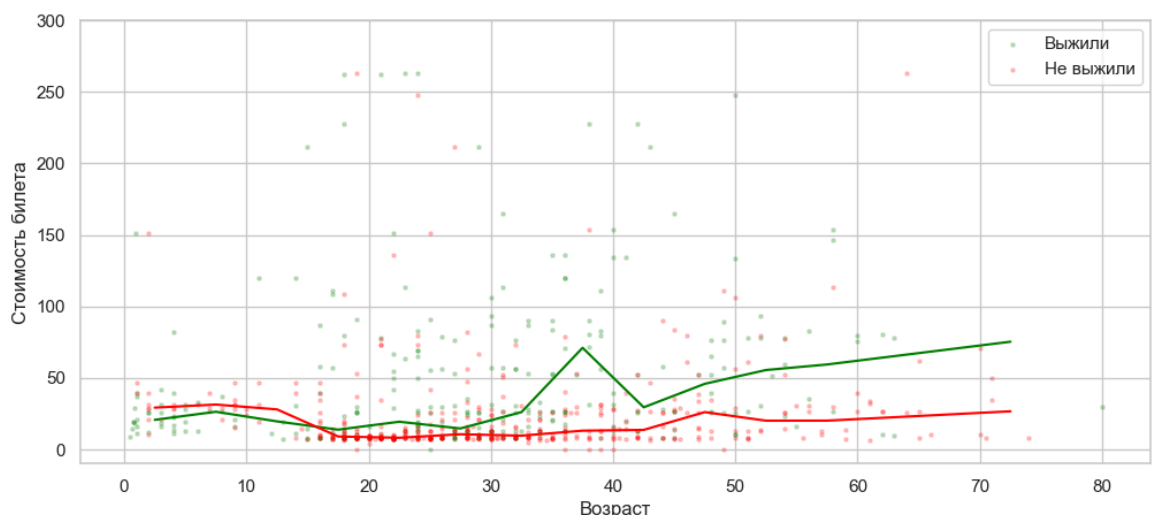
	group_age	survived	fare
0	2.5	0	29.125
1	2.5	1	20.575
2	7.5	0	31.275
3	7.5	1	26.250
4	12.5	0	27.900

Хорошая визуализация

```

Ввод [5]: 1 df_groupby_age_0 = df_groupby_age[df_groupby_age['survived'] == 0]
          2 df_groupby_age_1 = df_groupby_age[df_groupby_age['survived'] == 1]
          3
          4 plt.figure(figsize=(12, 5))
          5 plt.scatter(x=df_1['age'], y=df_1['fare'], c='green', s=5, label='He
          6 plt.scatter(x=df_0['age'], y=df_0['fare'], c='red', s=5, label='He
          7 plt.plot(df_groupby_age_1['group_age'], df_groupby_age_1['fare'],
          8 plt.plot(df_groupby_age_0['group_age'], df_groupby_age_0['fare'],
          9 plt.ylim((-10, 300))
          10 plt.xlabel('Возраст')
          11 plt.ylabel('Стоимость билета')
          12 plt.legend()
          13 plt.show()

```



3. Интерактивные графики в Plotly и Dash

3.1 Основы Plotly: создание интерактивных графиков

Ввод [11]:

```
1 import plotly.io as pio
2 import plotly.graph_objects as go
3 import plotly.express as px
4 pio.renderers.default='notebook'
```

Линейный график

Ввод [7]:

```
1 df_groupby_age['alone'] = df.groupby(['group_age', 'survived'])['alone'].count()
2 df_groupby_age['cnt'] = df.groupby(['group_age', 'survived'])['alone'].count()
3
4 df_groupby_age.head(5)
```

Out [7]:

	group_age	survived	fare	alone	cnt
0	2.5	0	29.125	0.000000	13
1	2.5	1	20.575	0.000000	27
2	7.5	0	31.275	0.000000	11
3	7.5	1	26.250	0.090909	11
4	12.5	0	27.900	0.222222	9

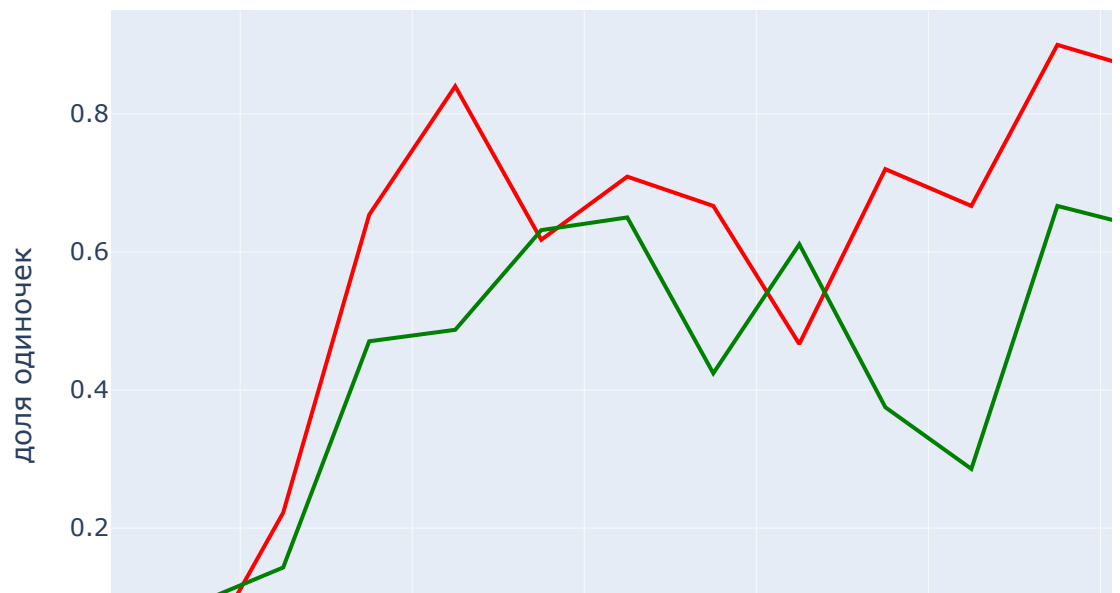
Ввод [8]:

```

1 # Создание фигуры
2 fig = go.Figure()
3
4 survived = df_groupby_age[df_groupby_age['survived'] == 1]
5 not_survived = df_groupby_age[df_groupby_age['survived'] == 0]
6 fig.add_scatter(x=not_survived['group_age'], y=not_survived['alone'],
7                 name='no')
8 fig.add_scatter(x=survived['group_age'], y=survived['alone'], mode=
9
10 # Настройка осей и заголовка
11 fig.update_layout(title="Зависимость доли одиночек от возраста",
12                   xaxis_title="возраст", yaxis_title="доля одиночек")
13
14 # Отображение
15 fig.show()
16 fig.write_html('linear.html')

```

Зависимость доли одиночек от возраста



Улучшим график, добавив доверительные интервалы по ЦПТ

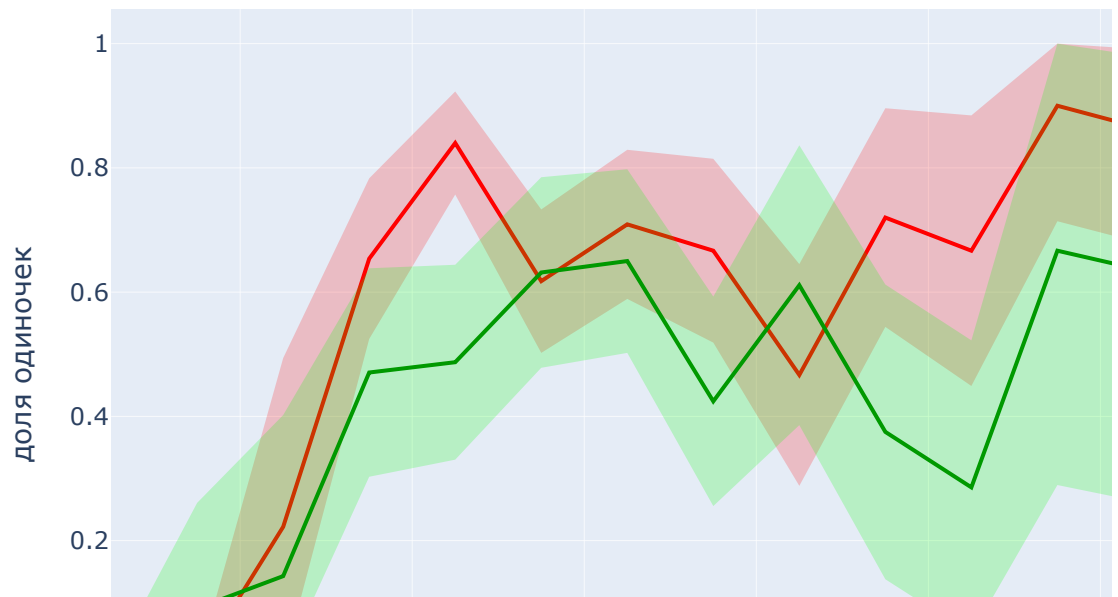
Ввод [9]:

```

1 alpha = 0.95
2 z = sps.norm.ppf((1 + alpha) / 2)
3
4 # Создание фигуры
5 fig = go.Figure()
6
7 survived = df_groupby_age[df_groupby_age['survived'] == 1]
8 not_survived = df_groupby_age[df_groupby_age['survived'] == 0]
9
10 y = not_survived['alone']
11 std = np.sqrt(y * (1 - y))
12 n = not_survived['cnt']
13
14 fig.add_scatter(x=not_survived['group_age'], y=y, mode='lines', marker=
15                 name='no')
16 fig.add_traces([go.Scatter(x=not_survived['group_age'], y=np.maximum(
17                             mode='lines', line_color = 'rgba(0,0,0,0.5)',
18                             showlegend=False),
19                     go.Scatter(x=not_survived['group_age'], y=np.minimum(
20                             mode='lines', line_color='rgba(0,0,0,0.5)',
21                             fill='tonexty', fillcolor = 'rgba(255,255,255,0.5)',
22                             showlegend=False))])
23 y = survived['alone']
24 std = np.sqrt(y * (1 - y))
25 n = survived['cnt']
26
27 fig.add_scatter(x=survived['group_age'], y=y, mode='lines', marker=
28                 name='yes')
29 fig.add_traces([go.Scatter(x=survived['group_age'], y=np.maximum(
30                             mode='lines', line_color='rgba(0,0,0,0.5)',
31                             showlegend=False),
32                     go.Scatter(x=survived['group_age'], y=np.minimum(
33                             mode='lines', line_color='rgba(0,0,0,0.5)',
34                             fill='tonexty', fillcolor='rgba(0, 255, 255, 0.5)',
35                             showlegend=False))])
36
37 # Настройка осей и заголовка
38 fig.update_layout(title="Зависимость доли одиночек от возраста",
39                   xaxis_title="возраст", yaxis_title="доля одиночек")
40
41 # Отображение
42 fig.show()
43 fig.write_html('linear_conf_int.html')

```

Зависимость доли одиночек от возраста



Вывод:

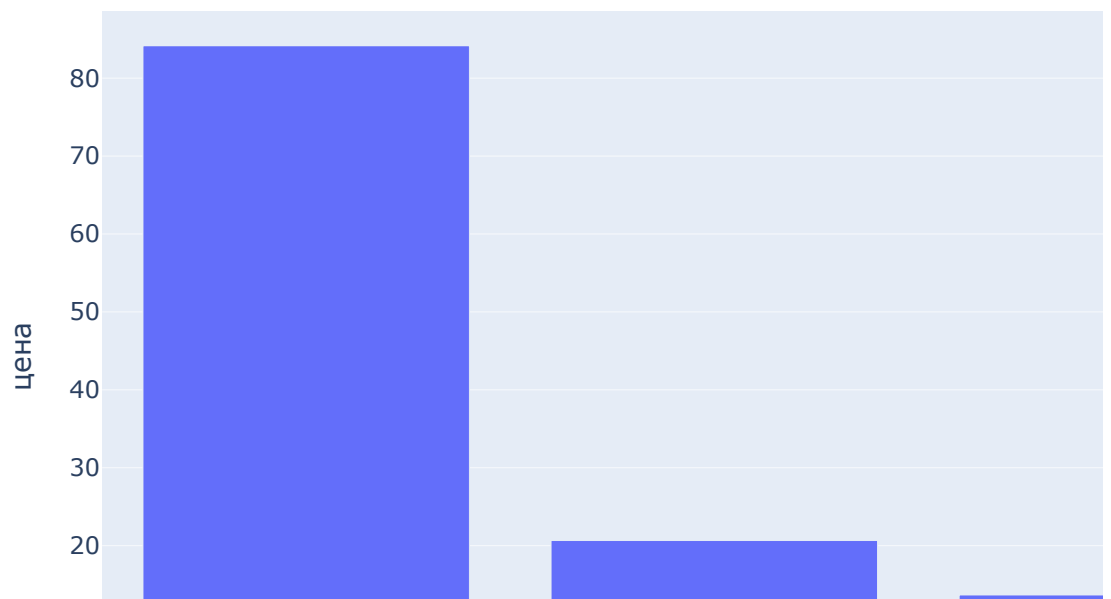
- выжившие незначимо чаще были не одни
- ожидаемо несовершеннолетние редко бывали одни на корабле. После 20 лет различия незаметны

Столбчатая диаграмма

Способ 1

```
Ввод [12]: 1 df_groupby_pclass = df.groupby('pclass')['fare'].mean().reset_index()
2
3 # Создание столбчатого графика
4 fig = px.bar(df_groupby_pclass, x='pclass', y='fare', title='Диаграмма цены от класса')
5
6 fig.update_layout(xaxis=dict(tickmode='linear', tick0=1, dtick=1),
7
8 fig.show()
9 fig.write_html('bar_1.html')
```

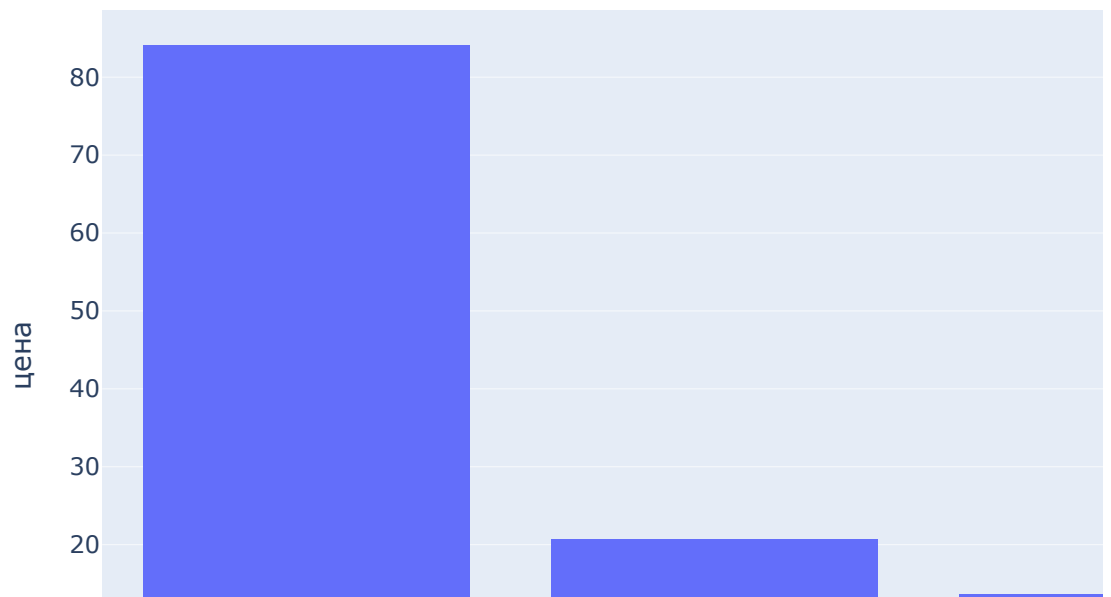
Диаграмма цены от класса



Способ 2

```
Ввод [13]: 1 df_groupby_pclass = df.groupby('pclass')['fare'].mean().reset_index()
2
3 # Создание столбчатого графика
4 fig = px.histogram(df, x='pclass', y='fare', histfunc='avg', title='Average Fare by Class')
5
6 fig.update_layout(xaxis=dict(tickmode='linear', tick0=1, dtick=1),
7                    bargap=0.2)
8
9
10 fig.show()
11 fig.write_html('bar_2.html')
```

Диаграмма цены от класса



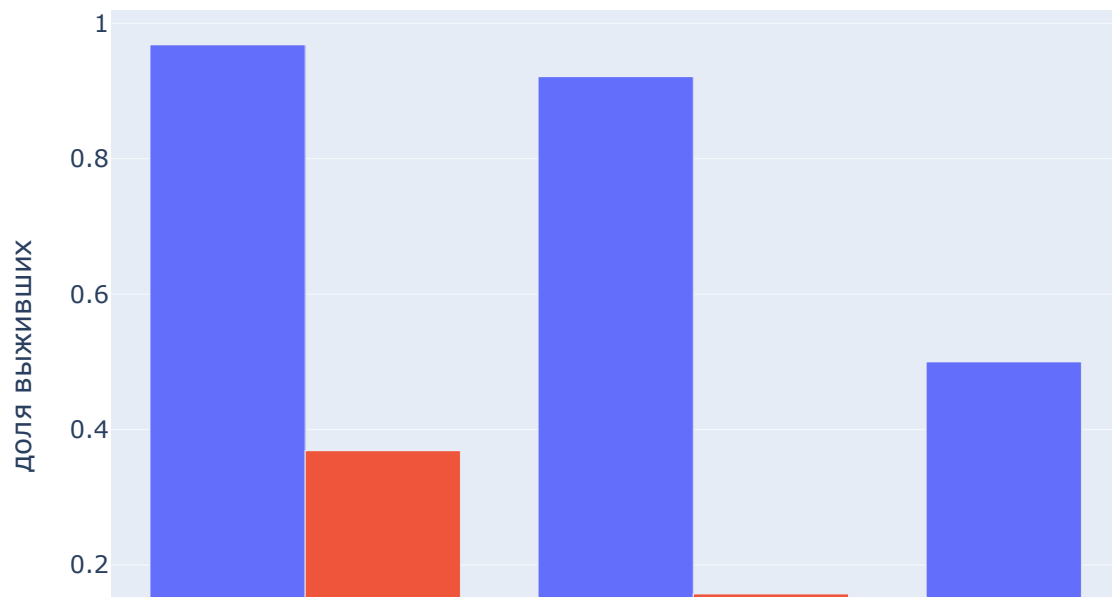
Практика: постройте гистограмму распределения возраста в зависимости от выживания и сделайте вывод

Способ 1

Ввод [14]:

```
1 # График выживаемости по классам обслуживания
2 df_groupby_pclass_sex = df.groupby(['pclass', 'sex'])['survived'].
3
4
5 fig_bar = px.bar(df_groupby_pclass_sex, x='pclass', y='survived',
6                  title='Выживаемость по классам')
7
8 fig_bar.update_layout(xaxis_title="класс", yaxis_title="доля выжи")
9
10 fig_bar.show()
11 fig_bar.write_html('bar_divided_1.html')
```

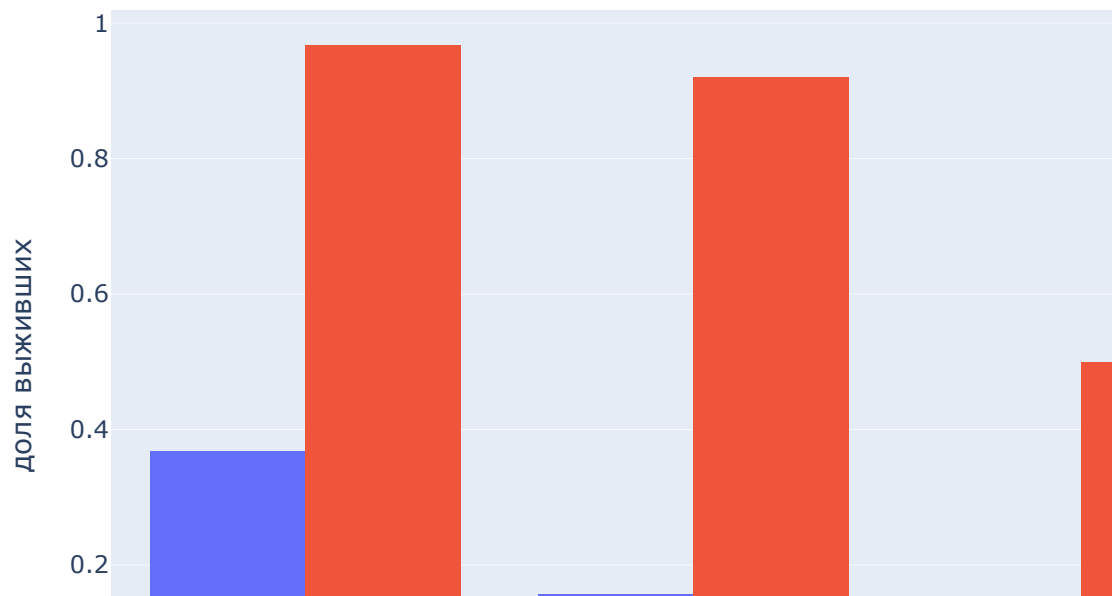
Выживаемость по классам



Способ 2

```
Ввод [15]: 1 fig_bar = px.histogram(df, x='pclass', y='survived', color='sex',
2                  title='Выживаемость по классам')
3 fig_bar.update_layout(xaxis_title="класс", yaxis_title="доля выживших")
4
5 fig_bar.show()
6 fig_bar.write_html('bar_divided_2.html')
```

Выживаемость по классам



Выводы:

- доля выживших женщин значительно больше, чем доля выживших мужчин. Это подтверждает тезис, что сначала спасали женщин и детей
- вероятность выжить у пассажиров 1 и 2 класса значительно больше, чем у пассажиров классом ниже

Waterfall-графики

Водопадные графики используются чаще всего для анализа итоговой прибыли компании: выявляются главные источники доходов и расходов и находятся сильные и слабые места в экономике продукта

Вернемся к кейсу с Yxxxxx.Такси

Построим waterfall-график экономики сервиса в месяц. Для этого нам нужны дополнительно данные о расходах:

- аренда авто компанией: 15000₽ в месяц за эконом, 20000₽ в месяц за комфорт и 25000₽ в месяц за комфорт+
- аренда парковки: 5000₽ в месяц за одно парковочное место
- ремонт автомобилей: 5 млн ₽ в месяц суммарно
- бонусные выплаты водителям: 20000₽ единовременно за 250 заказов в месяц
- аренда офиса для IT: 2 млн ₽ в месяц
- зарплата IT-специалистов: 7 млн ₽ в месяц суммарно
- затраты на рекламу: 7 млн ₽ в месяц суммарно
- прочие затраты: 5 млн ₽ в месяц суммарно

Посчитаем нужные нам величины и отобразим их на графике

Ввод [25]:

```
1 df = pd.read_csv('orders_group.csv')
2 df
```

Out [25]:

	taxist_id	date	orders_cnt	total_cost	total_service_fee	auto_id	tariff	rent_cost
0	1	2024-09-01	12	8674	1734.8	1494	эконом	1200
1	1	2024-09-02	10	8136	1627.2	1494	эконом	1200
2	1	2024-09-03	9	6361	1272.2	1494	эконом	1200
3	1	2024-09-04	13	9576	1915.2	1494	эконом	1200
4	1	2024-09-05	7	3664	732.8	1494	эконом	1200
...	...	...	...	...	...	...	...	...
32909	1609	2024-09-19	14	21130	4226.0	565	комфорт	2400
32910	1609	2024-09-22	8	10786	2157.2	565	комфорт	2400
32911	1609	2024-09-24	13	15341	3068.2	565	комфорт	2400
32912	1609	2024-09-25	8	6322	1264.4	565	комфорт	2400
32913	1609	2024-09-30	14	15372	3074.4	565	комфорт	2400

32914 rows × 10 columns

Ввод [48]:

```
1 # Доходы сервиса
2 df_group = df.groupby('tariff')[[
3     'orders_cnt', 'total_cost', 'total_service_fee',
4     'rent_cost', 'driver_income', 'company_income'
5 ]].sum().reset_index()
6 df_group
7 total_fee_income = df_group['total_service_fee'].sum()
8 total_rent_income = df_group['rent_cost'].sum()
9 total_fee_income, total_rent_income
```

Out [48]: (56265065.0, 61822450)

Ввод [55]:

```

1 # Расходы сервиса на авто
2
3 c_auto_econom = 15_000
4 c_auto_comfort = 20_000
5 c_auto_comfort_plus = 25_000
6 c_parking_per_one = 5_000
7 c_repair = 5_000_000
8
9 cars = pd.read_csv('cars_clean.csv')
10 cars['rent_company_cost'] = cars['tariff'].apply(
11     lambda t: c_auto_econom if t == 'эконом' else \
12     c_auto_comfort if t == 'комфорт' else c_auto_comfort_plus)
13 cars['parking_cost'] = c_parking_per_one
14 cars_costs = cars.groupby('tariff')[['rent_company_cost', 'parking_cost']].sum()
15 total_rent_company_cost = -cars_costs['rent_company_cost'].sum()
16 total_parking_cost = -cars_costs['parking_cost'].sum()
17 total_repair_cost = -c_repair
18 total_rent_company_cost, total_parking_cost, total_repair_cost

```

Out[55]: (-32815000, -9065000, -5000000)

Ввод [56]:

```

1 # Расходы сервиса на бонусы водителям
2
3 cnt_orders_bonus = 250
4 bonus_amt = 20_000
5
6 df_driver = df.groupby('taxist_id')[['orders_cnt']].sum()
7 df_driver['bonus'] = df_driver['orders_cnt'].apply(
8     lambda cnt: 0 if cnt < cnt_orders_bonus else bonus_amt
9 )
10 total_drivers_bonuses_costs = -df_driver['bonus'].sum()
11 total_drivers_bonuses_costs

```

Out[56]: -3620000

Ввод [57]:

```

1 # Остальные расходы сервиса
2
3 total_office_cost = -2_000_000
4 total_it_salary_cost = -7_000_000
5 total_ad_costs = -7_000_000
6 total_another_costs = -5_000_000

```

Построим waterfall-график

Ввод [68]:

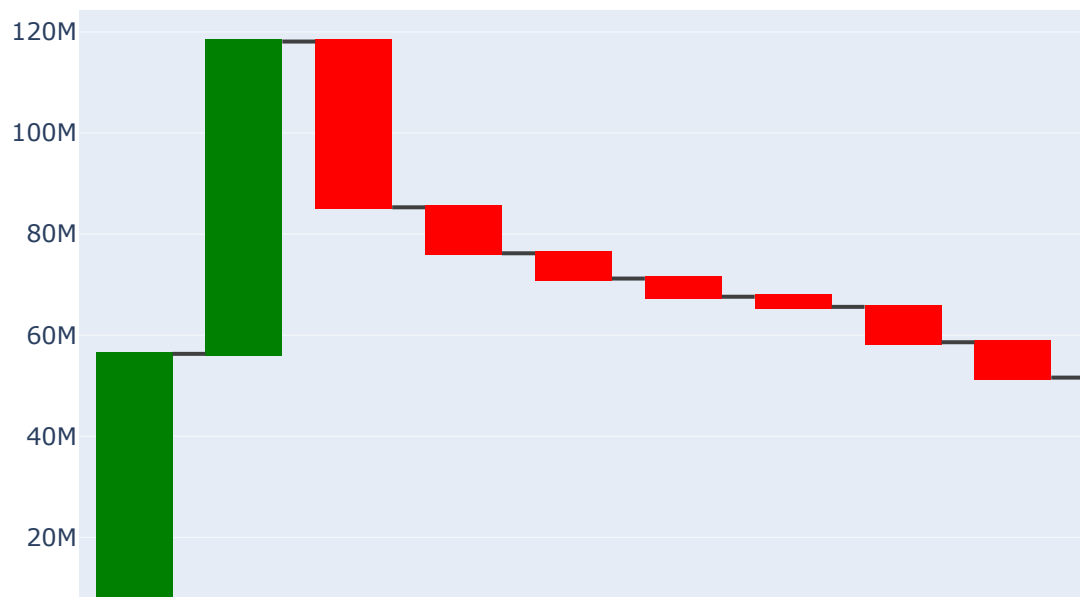
```

1  # Исходные значения
2  values = [
3      total_fee_income,
4      total_rent_income,
5      total_rent_company_cost,
6      total_parking_cost,
7      total_repair_cost,
8      total_drivers_bonuses_costs,
9      total_office_cost,
10     total_it_salary_cost,
11     total_ad_costs,
12     total_another_costs,
13     0 # total – Plotly считает сам
14 ]
15 values = np.round(values, -5)
16
17 fig = go.Figure(go.Waterfall(
18     name = "2024",
19     orientation = "v",
20     measure = [
21         "relative", # доход комиссии
22         "relative", # доход аренда авто
23         "relative", # расходы аренда авто
24         "relative", # расходы парковка
25         "relative", # расходы ремонт
26         "relative", # расходы бонусы водителям
27         "relative", # расходы офис
28         "relative", # расходы IT зарплата
29         "relative", # расходы реклама
30         "relative", # расходы прочие
31         "total"     # итог
32     ],
33     x = [
34         "Доход: комиссия",
35         "Доход: аренда авто",
36         "Расход: аренда авто",
37         "Расход: парковка",
38         "Расход: ремонт авто",
39         "Расход: бонусы водителям",
40         "Расход: офис",
41         "Расход: IT зарплата",
42         "Расход: реклама",
43         "Расход: прочие",
44         "Итоговая прибыль"
45     ],
46     textposition = "outside",
47     y = values,
48     # text = text_labels, # ← добавляем подписи
49     connector = {"line":{"color":"rgb(63, 63, 63)"}}},
50     increasing = {"marker":{"color":"green"}}},
51     decreasing = {"marker":{"color":"red"}}},
52     totals = {"marker":{"color":"blue"}}},
53 ))
54
55 fig.update_layout(
56     title = "Waterfall: экономика сервиса за месяц",
57     waterfallgap = 0.3,
58 )
59
60 fig.show()

```

```
61 fig.write_html("economics.html") # Сохраняем в HTML
```

Waterfall: экономика сервиса за месяц



Heatmap-графики

Heatmap используются для анализа совместного распределения двух признаков. С помощью тепловых карт можно выявить корреляцию между признаками, оценить совместное распределение и разбить данные на сегменты. Обычно heatmap строят, когда хотят найти зависимости ключевых метрик друг от друга либо для первичного анализа данных

Ввод [69]:

```
1 orders = pd.read_csv('orders_clean.csv')
2 orders
```

Out [69]:

	id	data	taxist	client	r1	r2	tariff	len	time	coef	cost	aut
0	1	2024-09-16	151.0	1.0	NaN	5.0	эконом	10.0000	19.00	1.09	293	44
1	2	2024-09-08	511.0	1.0	NaN	5.0	эконом	11.7000	22.00	0.92	286	147
2	3	2024-09-14	632.0	1.0	NaN	5.0	эконом	14.1600	18.14	1.27	387	71
3	4	2024-09-03	1439.0	2.0	NaN	5.0	комфорт	27.6000	45.00	1.37	1597	124
4	5	2024-09-19	292.0	2.0	5.0	NaN	комфорт	44.6000	72.33	0.92	1737	164
...	...	...	...	...	...	...	...	...	...	...	...	...
341304	341305	2024-09-21	1288.0	97566.0	NaN	5.0	комфорт	37.8600	65.30	1.21	2010	24
341305	341306	2024-09-17	1421.0	97567.0	5.0	5.0	эконом	59.8000	84.50	1.10	1494	109
341306	341307	2024-09-09	680.0	97567.0	NaN	5.0	эконом	20.2400	40.20	1.28	722	175
341307	341308	2024-09-01	1039.0	97567.0	NaN	5.0	эконом	6.0000	9.00	1.30	179	32
341308	341309	2024-09-05	584.0	97567.0	5.0	NaN	эконом	26.1178	34.40	0.85	485	155

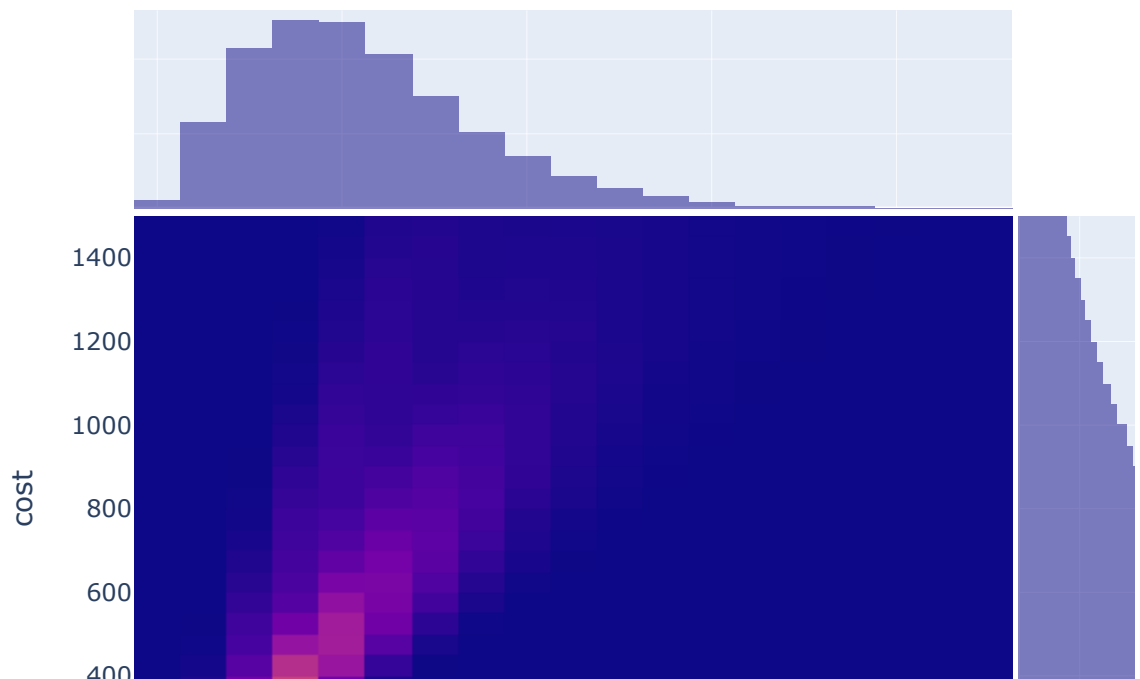
341309 rows × 13 columns

```

Ввод [83]: 1 orders_small = orders[orders['cost'] < 1_500]
           2
           3 fig = px.density_heatmap(
           4     orders_small,
           5     x="len",
           6     y="cost",
           7     nbinsx=30,
           8     nbinsy=30,
           9     marginal_x="histogram", # маргинальный график по X
          10     marginal_y="histogram", # маргинальный график по Y
          11 )
          12
          13 fig.update_layout(
          14     title="Heatmap с маргинальными распределениями: Длина поездки
          15 )
          16
          17 fig.show()

```

Heatmap с маргинальными распределениями: Длина поездки



3.2 Введение в Dash

Интерактивный график с карт

```
1 !pip install dash
```

Ввод [84]:

```
1 # Загружаем датасет с данными цен квартир (запусти, если выполняе
2 df = pd.read_csv('input_data.csv', sep=';', decimal=',')
3 df['geo_lat'] = df['geo_lat'].astype('float')
4 df['geo_lon'] = df['geo_lon'].astype('float')
5
6 # Загружаем датасет с регионами
7 regions = pd.read_excel('regions.xlsx')
8 regions['id_region'] = regions['GNINMB,C,4'] // 100
9 regions['name_region'] = regions['NAME,C,40']
```


Ввод [88]:

```

1 import dash
2 from dash import dcc, html, Input, Output
3
4
5 # Функция для агрегации данных по количеству комнат
6 def aggregate_data(rooms=None):
7     filtered_df = df if rooms is None else df[df['rooms'] == rooms]
8     metro_summary = filtered_df.groupby("id_region").agg(
9         count=("price", "count"),
10        median_price=("price", "median"),
11        lat=("geo_lat", "median"),
12        lon=("geo_lon", "median")
13    ).reset_index()
14    metro_summary['cost'] = metro_summary['median_price'].apply(lambda x: x * 1.5)
15    return metro_summary.merge(regions[['id_region', 'name_region']])
16
17
18 # Создаем дашборд
19 app = dash.Dash(__name__)
20 app.layout = html.Div([
21     html.H1("Карта цен на квартиры в России по регионам"),
22     html.Label("Выберите количество комнат:"),
23     dcc.Dropdown(
24         id='room-filter',
25         options=[
26             {'label': 'Все', 'value': 'all'}
27         ] + [{'label': f'{i} комн.', 'value': i} for i in sorted(df['rooms'].unique())],
28         value='all',
29         clearable=False
30     ),
31     dcc.Graph(id="russia_map")
32 ])
33
34 @app.callback(
35     Output("russia_map", "figure"),
36     Input("room-filter", "value")
37 )
38 def update_map(selected_rooms):
39     rooms = None if selected_rooms == 'all' else int(selected_rooms)
40     metro_summary_region = aggregate_data(rooms)
41     fig = px.scatter_mapbox(
42         metro_summary_region,
43         lat="lat",
44         lon="lon",
45         size="count",
46         color="median_price",
47         hover_name="name_region",
48         title="Стоимость квартир и количество предложений по регионам",
49         color_continuous_scale="Bluered",
50         size_max=20,
51         center={"lat": 55, "lon": 55}, # Центрируем карту
52         zoom=3,
53         mapbox_style="open-street-map"
54     )
55     fig.write_html("map.html") # Сохраняем в HTML — там можно будет открыть в браузере
56     return fig
57
58 app.run_server(debug=True, port=8051)

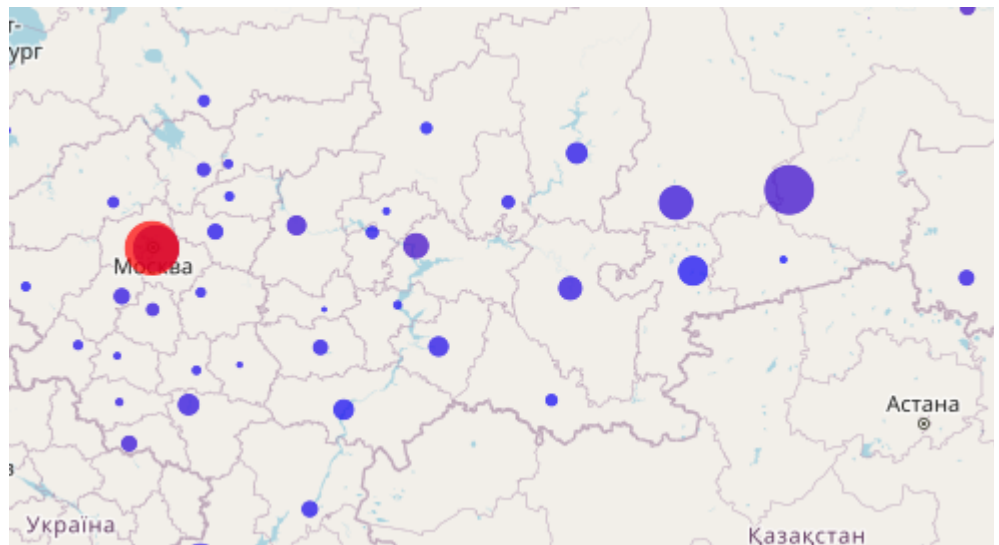
```

Карта цен на квартиры в России по регионам

Выберите количество комнат:

Все

Стоимость квартир и количество предложений по регионам



4. Фреймворк STAR

Вопрос в зал: что такое фреймворк STAR?

Фреймворк STAR (Situation, Task, Action, Result) — это метод структурированного представления информации, который помогает четко и логично излагать идеи. В контексте визуализации данных он может использоваться для структурирования анализа, презентации выводов и объяснения методологии.

Пример использования: я поискал на habr'е информацию про фреймворк STAR и обнаружил в топе страниц следующие:

- [Как подготовиться к собеседованию в зарубежных компаниях по методике STAR и почему это не очередная выдумка HR](https://habr.com/ru/companies/netologyru/articles/691510/) (<https://habr.com/ru/companies/netologyru/articles/691510/>)
- [Как я перестал превращать собес в экзамен: оцениваем хард- и софт-скиллы за одно собеседование](https://habr.com/ru/companies/plesk/articles/552960/) (<https://habr.com/ru/companies/plesk/articles/552960/>)

Как видите, этот фреймворк довольно популярен в подготовке и проведении собеседования

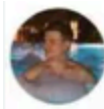
Формат STAR:

1. **Situation (Ситуация):** Контекст проблемы или задачи.
2. **Task (Задача):** Конкретная цель, которую нужно достичь.
3. **Action (Действие):** Шаги, предпринятые для решения задачи.
4. **Result (Результат):** Итог выполненных действий.

Кейс: Борьба с зависанием в соцсетях



- **S (Ситуация):** Я заметил, что слишком много времени провожу в соцсетях, из-за чего не успеваю делать важные дела. (рабочие задачи, учёба, домашние дела, хобби, etc)
- **T (Задача):** Я поставил перед собой задачу понять, сколько времени уходит на соцсети и как это влияет на продуктивность.
- **A (Действие):** Я начал отслеживать время использования телефона. Понял, что в дни зависания в соцсетях у меня не хватало времени на другие активности, и я просрачивал дедлайны или у меня не было времени на мои хобби.
- **R (Результат):** После проведенного анализа я решил поставить лимиты на использование некоторых приложений. В итоге свободного времени стало больше, а дела делаются быстрее. Правда, есть один минус: теперь SLA моих ответов стал немного больше. 😊



Артём Трещенко
@treschenko24

Читать

Когда все вокруг говорят про финансовую грамотность, а ты просто чилловый парень, который тратит деньги так, как чувствует



- **S (Situation) — Ситуация:** Около года назад я понял, что достаточно много трачу: несмотря на солидную зарплату накоплений практически не было.
- **T (Task) — Задача:** Я поставил перед собой цель — научиться лучше управлять своими деньгами, создать подушку безопасности и начать инвестировать, чтобы накопить на квартиру.
- **A (Action) — Действия:** Я начал с малого: завёл таблицу расходов и стал фиксировать каждую трату. Затем прошёл онлайн-курс по инвестициям, где научился разбираться в базовых инвестиционных инструментах. Также я открыл

5. 📌 Практика: презентация результатов по STAR

Студенты разбиваются на 4 группы. Каждая группа получает ситуацию и визуализацию. Каждая группа должна додумать, что они сделали в кейсе, что получили и что предлагают изменить в продукте

Подготовка студентов и короткое выступление по STAR

Обратная связь от участников и семинариста

Кейс 1. Проседание покупок

Ситуация: В мобильном приложении для заказа еды изменили экран онбординга — теперь регистрация обязательна перед показом каталога. После обновления аналитик проверяет воронку покупок: из-за обязательной регистрации часть пользователей ушла с воронки, однако мы получили прирост в регистрациях, то есть, больше информации о новых пользователях

Задача студентов: Использовать фреймворк **STAR**, чтобы разобрать:

- S: Почему изменился онбординг
- T: Что хотим понять (влияние на конверсию)
- A: Что делаем (анализ воронки)
- R: Что решаем (гипотезы про упрощение регистрации)

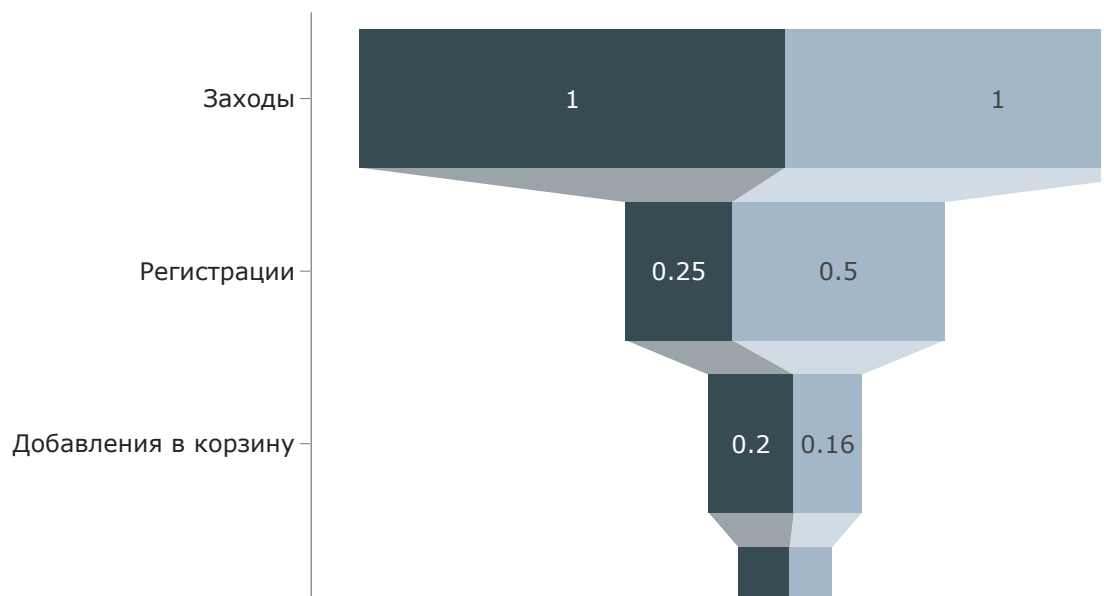
График:

```

Ввод [102]: 1 data = dict(Quantity=[1, 0.25, 0.2, 0.12,
2              1, 0.5, 0.16, 0.10],
3              Stage=['Заходы', 'Регистрации', 'Добавления в корзину',
4              Process=['Старый']*4 + ['Новый']*4)
5
6 fig = px.funnel(data, y='Stage', x='Quantity', color='Process',
7               color_discrete_map={"Старый": "#374B53",
8               "Новый": "#A4B7C8"},
9               template="simple_white",
10              title='Добавление обязательной регистрации',
11              labels={"Stage": ""})
12 fig.show()

```

Добавление обязательной регистрации



Кейс 2. Резкий рост оттока

Ситуация: Подписной сервис (онлайн-курсы) запустил акцию: первый месяц подписки с большой скидкой. Retention пользователей новой акции оказался хуже, однако мы привлекли новых пользователей. Что делать?

Задача студентов: Использовать **STAR**, чтобы разобраться:

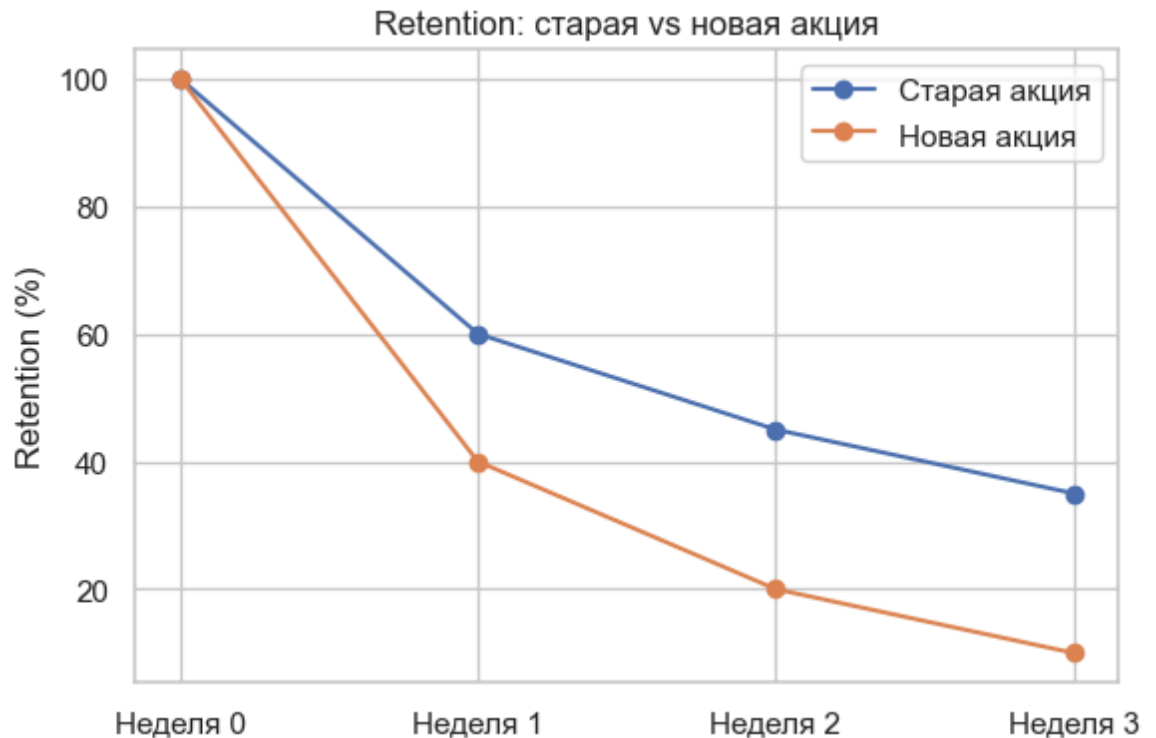
- S: Акция → много новых пользователей
- T: Проверить поведение (retention, LTV)
- A: Сравнить когортные графики
- R: Принять решение об изменении условий или сегментировании

Ввод [103]:

```

1 weeks = ['Неделя 0', 'Неделя 1', 'Неделя 2', 'Неделя 3']
2 old = [100, 60, 45, 35]
3 new = [100, 40, 20, 10]
4
5 plt.figure(figsize=(6, 4))
6 plt.plot(weeks, old, marker='o', label='Старая акция')
7 plt.plot(weeks, new, marker='o', label='Новая акция')
8 plt.title('Retention: старая vs новая акция')
9 plt.ylabel('Retention (%)')
10 plt.legend()
11 plt.tight_layout()

```



Кейс 3. Локальное падение NPS

Ситуация: Сервис доставки продуктов отслеживает NPS по городам. В одном городе NPS резко упал за неделю. Что могло пойти не так?

Задача студентов: Использовать **STAR**, чтобы разобрать:

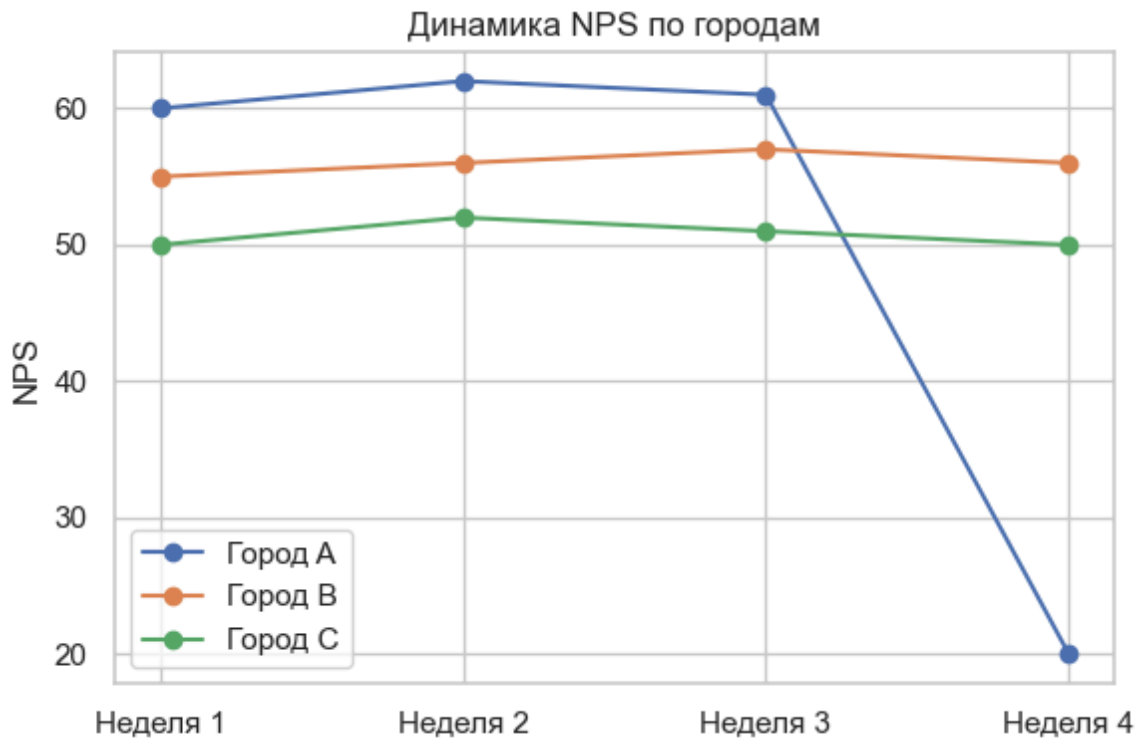
- S: Локальное падение
- T: Найти причину
- A: Проверить жалобы, сроки доставки, партнёров
- R: Выработать решение (замена партнёра, компенсации)

График:


```

Ввод [104]: 1 weeks = ['Неделя 1', 'Неделя 2', 'Неделя 3', 'Неделя 4']
2 city_a = [60, 62, 61, 20]
3 city_b = [55, 56, 57, 56]
4 city_c = [50, 52, 51, 50]
5
6 plt.figure(figsize=(6, 4))
7 plt.hist(weeks, city_a, marker='o', label='Город А')
8 plt.plot(weeks, city_b, marker='o', label='Город В')
9 plt.plot(weeks, city_c, marker='o', label='Город С')
10 plt.title('Динамика NPS по городам')
11 plt.ylabel('NPS')
12 plt.legend()
13 plt.tight_layout()

```



Кейс 4. Тест нового сценария оплаты

Ситуация: Интернет-магазин протестировал новую кнопку «Оплатить позже». Ожидали рост конверсии, но она упала.

Задача студентов: Использовать **STAR**, чтобы разобрать:

- S: Новый сценарий оплаты
- T: Проверить эффект
- A: A/B тест → анализ разницы
- R: Принять решение — откатить или улучшить сценарий

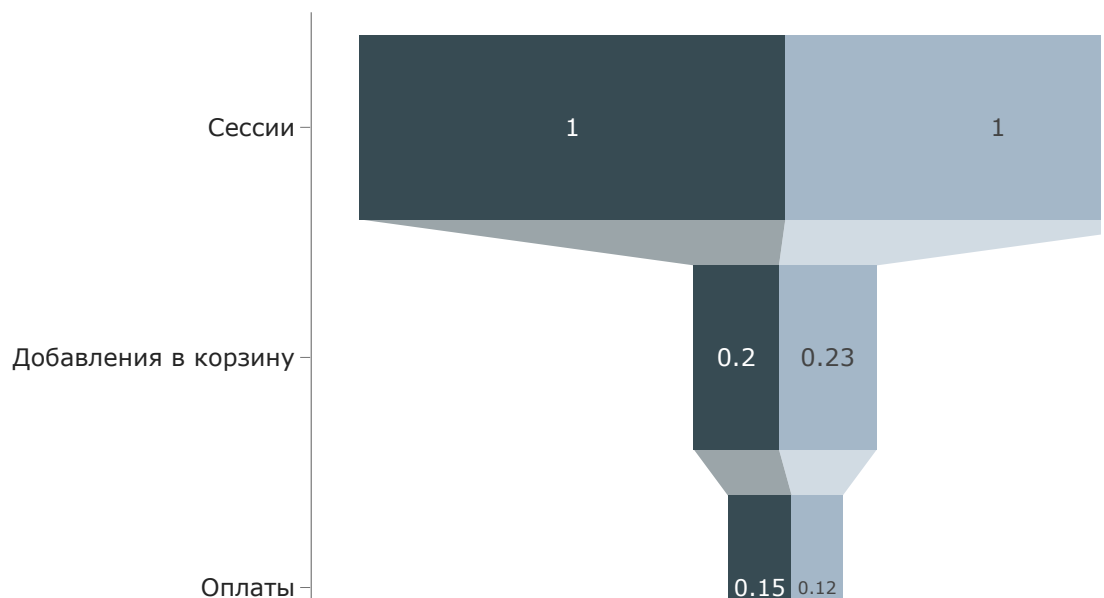
График:

```

Ввод [98]: 1 data = dict(Quantity=[1, 0.2, 0.15,
2             1, 0.23, 0.12],
3             Stage=['Сессии', 'Добавления в корзину', 'Оплаты']*2,
4             Process=['Старый']*3 + ['Новый']*3)
5
6 fig = px.funnel(data, y='Stage', x='Quantity', color='Process',
7                 color_discrete_map={"Старый": "#374B53",
8                                     "Новый": "#A4B7C8"},
9                 template="simple_white",
10                title='Новый сценарий оплаты',
11                labels={"Stage": ""})
12 fig.show()

```

Новый сценарий оплаты



6. Итоги семинара

С чем выходим с занятия:

- повторили, как нужно и как не нужно графически изображать данные
- научились интерактивным визуализациям с помощью plotly и dash
- изучили фреймворк STAR
- потренировались презентовать результаты перед заказчиком

7. Полезные материалы

- [Статья с плохими визуализациями \(https://www.luzmo.com/blog/bad-data-visualization\)](https://www.luzmo.com/blog/bad-data-visualization)
- [Еще одна статья про плохие визуализации \(https://habr.com/ru/articles/446880/\)](https://habr.com/ru/articles/446880/)
- [ТГ-канал "отвратительные графики" \(https://t.me/awfulcharts\)](https://t.me/awfulcharts)